Sprint 2 – Week 3 Closeout Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The platform enables:

- Organizations to post structured, well-defined micro-projects.
- Students to browse and discover opportunities.
- Students to apply for projects.
- Organizations to manage and review applications.
- A controlled workflow to manage project participation.

The system architecture follows a layered full-stack model:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

The platform supports two primary user roles:

- Student
- Organization

1.2 Sprint 2 Context

Sprint 2 expands the lifecycle of an application from:

- ➢ Submission → Visibility → Decision → Status Transition

Week 1 implemented:

- Application submission (US-6)
- Student-side status visibility (US-8)

Week 2 extended functionality to:

- Organization-side application review (US-7)

- Accept/Reject decision workflow (US-9)
- Frontend organization dashboard integration

Week 3 finalized Sprint 2 by completing:

- Status synchronization (US-8 refinement)
- Business rule enforcement (single accepted application per project)
- End-to-End workflow validation
- Regression hardening
- Test results reporting & coverage verification
- Sprint retrospective

1.3 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10

FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 2 – Week 3 Overview

2.1 Sprint Goal

The goal of Sprint 2 – Week 3 was to finalize and harden the application decision workflow, ensuring that business rules, system integrity, and user interface synchronization were fully enforced before closing Sprint 2.

While Week 1 introduced application submission and Week 2 implemented the Accept/Reject decision endpoint and dashboard interface, Week 3 ensured that:

- Only one application per project can be accepted
- Status updates propagate correctly across dashboards
- Transition rules are strictly enforced
- All workflows pass system-level validation
- Test coverage meets quality standards

This sprint marked the transition from feature development to stability, enforcement, and production-level readiness.

3. Sprint Planning

3.1 Groomed Product Backlog

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern

3.2 Sprint Backlog – Week 2 Commitment

Week 3 served as the stabilization and validation phase of Sprint 2. Unlike Weeks 1 and 2, which were feature-heavy, Week 3 focused on enforcing business rules, strengthening system integrity, and validating complete workflow behavior.

Issue	Title		Type
#55	**Enforce Single Accepted Applicant Rule**	**3**	**Backend**
#56	**Sync Student Dashboard Status Updates**	**3**	**Frontend**
#57	**Full Integration & Regression Testing**	**5**	**System Validation**

Total Commitment: 11 Story Points

Commitment Rationale

Week 3 commitments were selected strategically to finalize Sprint 2 functionality and ensure production-level correctness before sprint closure.

Issue #55 – Enforce Single Accepted Applicant Rule (3 SP)

Why it was committed:

Although Accept/Reject functionality was implemented in Week 2, the system did not enforce exclusivity. Multiple applications could technically be marked as accepted for the same project, violating core business logic.

This issue was prioritized because:

- It protects data integrity.
- It enforces real-world hiring constraints.
- It prevents inconsistent project states.
- It strengthens backend rule enforcement.

Scope of Work Included:

- Modify PATCH /applications/{id}/status logic.
- Add automatic rejection of remaining pending applications when one is accepted.
- Add transition validation logic.

- Add unit and integration tests.

Complexity Justification (3 SP):
Moderate complexity. Required:

- Careful querying logic
- Transaction handling
- Edge case validation
- Additional test coverage

Issue #56 – Sync Student Dashboard Status Updates (3 SP)

Why it was committed:

After backend transition enforcement, the frontend dashboard needed to reflect updated statuses instantly. Previously, status changes required manual refresh or did not propagate cleanly across dashboards.

This issue ensured:

- UI consistency with backend state
- Immediate visual feedback
- Disabled action buttons after decision
- Reduced user confusion

Scope of Work Included:

- React state management refinement
- Conditional rendering updates
- Badge color standardization
- Button disable logic after transition

Complexity Justification (3 SP):
Moderate complexity due to frontend-backend integration and state synchronization, though no new API endpoints were introduced.

Issue #57 – Full Integration & Regression Testing (5 SP)

Why it was committed:

Sprint 2 introduced multi-role interaction:

- Students submit applications.
- Organizations review and decide.
- Status updates propagate back to students.

This issue validated the entire lifecycle from creation to decision enforcement.

Scope of Work Included:

- Full scenario testing (manual + automated)
- Integration testing across routers
- Regression testing for prior functionality
- Coverage verification
- Error-handling validation

Complexity Justification (5 SP):
Higher complexity because:

- Cross-module validation required
- Multiple roles involved
- State transitions tested across dashboards
- Required refinement of test fixtures

This was appropriately sized at 5 points due to system-wide validation impact.

Capacity & Workload Analysis

Week 3 capacity was intentionally reduced compared to Week 1 and Week 2 because:

- Core feature development was completed.
- Focus shifted toward hardening and validation.
- Business rule enforcement requires deeper reasoning but fewer interface additions.

The 11-point commitment reflects:

- Sustainable workload
- Realistic time allocation
- Focus on quality over feature quantity

The sprint concluded with:

- No spillover work
- No added scope mid-week
- All acceptance criteria satisfied

3.3 Sprint 2 Backlog Structure

Sprint 2 was divided into three weekly development phases.

Week 1 – Application Creation (Student Side)

Primary Focus:

- Allow students to apply to projects
- Provide application visibility
- Prevent duplicate submissions
- Integrate frontend UI workflow

Completed Issues

Issue	Description	Story Points	Status
#45	Create Application Database Model	3	Done
#46	Implement Apply to Project Endpoint	5	Done
#47	Implement Get My Applications Endpoint	3	Done
#49	Frontend Apply Button Integration	3	Done
#50	Student Applications Dashboard	3	Done

Week 1 Total Commitment: 17 Story Points

Week 2 – Organization Review Workflow

Primary Focus:

- Application visibility for organizations
- Status decision workflow

Completed Issues

Issue	Description	Story Points	Status
#48	Organization View Applications Endpoint	5	Done
#53	Accept / Reject Endpoint	5	Done
#54	Organization Dashboard Integration	3	Done

Week 2 Total Commitment: 13 Story Points

Week 3 — Workflow Completion & Stability

Primary Focus:

- Status synchronization
- Business rule enforcement
- Integration testing

Planned Issues

Issue	Description	Story Points	Status
#56	Status Badge Sync	3	Done
#55	Prevent Multiple Accepted Applications	3	Done
#57	End-to-End Workflow Testing	5	Done

Week 3 Total Commitment: 11 Story Points

4. Acceptance Criteria

4.1 US-8 – Status Badge Synchronization

Acceptance Criteria:

- Application status changes must be reflected immediately in the Student Applications Dashboard.
- Status badges must update automatically when the backend status changes.
- Status color coding must follow system UI conventions:
  - Pending → Yellow badge
  - Accepted → Green badge
  - Rejected → Red badge
- The dashboard must display the latest application status retrieved from the backend API.
- UI must update without requiring a manual browser refresh.
- Accept/Reject decisions performed by organizations must propagate to the student dashboard.
- Status badges must remain consistent across all interfaces, including:

    - Student dashboard
    - Organization dashboard
- Status badge display must not expose internal database identifiers.
- Returns:
    - 200 when applications are successfully retrieved.
- Frontend Requirements:
    - Status badges must be visually consistent across dashboards.
    - Application status must update dynamically when the organization decision occurs.
    - No page reload required for updated status visibility.
    - Color coding must clearly communicate the application state.

4.2 US-9 Enhancement – Prevent Multiple Accepted Applications

Acceptance Criteria:

- The system must enforce the rule that only one application can be accepted per project.
- When an organization accepts an application:
    - All remaining applications for the same project with status = "pending" must automatically be updated to "rejected".
    - The system must enforce this rule server-side, independent of frontend behavior.
    - Accepted applications must remain immutable once status changes from pending → accepted.
    - Organizations must not be able to accept additional applications once one has already been accepted.
- System must validate application state transitions before applying updates.
- Valid transitions remain:
    - pending → accepted
    - pending → rejected
- Invalid transitions:
    - accepted → rejected
    - rejected → accepted
    - accepted → accepted
    - rejected → rejected
- System must ensure database consistency across all applications linked to the project.

- Returns:
  - 200 on successful status update.
  - 400 if invalid status transition attempted.
  - 403 if user role is not organization.
  - 403 if organization does not own the associated project.
  - 404 if application not found.
- Frontend Requirements:
  - Once an application is accepted, all other applications must immediately display status rejected.
  - Action buttons must be disabled after decision completion.
  - UI must remain synchronized with backend logic.

4.3 Integration Workflow Validation

Acceptance Criteria:

- The system must successfully support the full application lifecycle workflow.
- The following end-to-end scenario must execute successfully:
  - Student browses available projects.
  - Student submits an application.
  - Application is stored with status = **pending**.
  - Organization retrieves applications for its project.
  - Organization reviews application list.
  - Organization accepts or rejects an application.
  - Status updates persist in the database.
  - Student dashboard reflects updated application status.
- System must prevent invalid transitions or unauthorized actions during the workflow.
- Regression testing must confirm that Sprint 1 functionality remains unaffected, including:
  - Authentication
  - Project browsing
  - Application submission
- All integration tests must execute successfully without errors.
- System test coverage must remain ≥ 95%.

5. Definition of Done (Sprint 2)

The Definition of Done was refined for Sprint 2 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- API contract documented.
- State transitions defined.

Implementation

- Role-based access control enforced.
- Ownership validation implemented.
- Pagination implemented (US-7).
- Status transition logic implemented (US-9).

Testing

- Unit tests written.
- Edge case validation included.
- Minimum 3 test cases per story.
- Coverage ≥ 95%.

Integration

- Frontend integration functional
- No regression to Week 1 & 2 features
- PR merged into main branch.

5.1 Definition of Done (Sprint 2 – Week 3)

The Definition of Done for Week 3 extends the baseline criteria established earlier in the sprint and focuses on workflow stability, system integrity, and integration validation.

A User Story is considered complete when the following conditions are satisfied.

Design

- Business rule enforcing single accepted application per project is formally defined.

- Status synchronization behavior between organization and student dashboard is documented.
- Full workflow lifecycle from application submission to status update is validated.

Implementation

- Backend logic enforces single accepted application rule.
- Pending applications automatically transition to rejected when one application is accepted.
- Frontend dashboards display status badges using consistent color coding.
- UI components update dynamically following backend status updates.
- All role-based access control rules remain enforced.
- Status transition logic prevents invalid updates.

Testing

- Unit tests written.
- Valid status transitions.
- Invalid status transitions.
- Automatic rejection of competing applications.
- Integration tests validate:
    - End-to-end application workflow.
    - Cross-role interactions.
- Regression tests confirm:
    - Sprint1 features remain functional.
    - Test coverage remains ≥ 95%.

Integration

- Frontend dashboards properly reflect backend state.
- Organization actions update student dashboard statuses.
- Status badge updates occur without page reload.
- No regressions introduced in previously implemented endpoints.
- All Sprint 2 issues successfully merged into the main branch.

Completion Criteria

- All Week 3 issues moved to Done in the project board.
- All tests passing.
- Pull requests reviewed and merged.

- Sprint board updated to reflect completed work.
- Sprint 2 declared complete.

6. Software Testing

6.1 Requirement-to-Test Mapping

The following traceability matrix maps **Functional Requirements → User Stories → Test Cases** implemented throughout **Sprint 2**. This ensures that each system requirement introduced during the sprint is validated through structured testing.

The matrix also demonstrates incremental expansion of test coverage across:

- Application submission workflow (Week 1)
- Organization review workflow (Week 2)
- Workflow completion and business rule enforcement (Week 3)

Sprint 2 – Week 1 Tests (Application Submission)

Functional Requirement	User Story	Test Case ID
FR-7	US-6	TC-US6-01
FR-7	US-6	TC-US6-02
FR-7	US-6	TC-US6-03
FR-7	US-6	TC-US6-04
FR-7	US-6	TC-US6-05
FR-8	US-8	TC-US8-01
FR-8	US-8	TC-US8-02

These tests validated the student application workflow, including:

- Application creation
- Duplicate application prevention
- Retrieval of student applications
- Status visibility within the student dashboard

Sprint 2 – Week 2 Tests (Organization Review Workflow)

Functional Requirement	User Story	Test Case ID
FR-8	US-7	TC-US7-01

FR-8	US-7	TC-US7-02
FR-8	US-7	TC-US7-03
FR-9	US-9	TC-US9-01
FR-9	US-9	TC-US9-02
FR-9	US-9	TC-US9-03
FR-9	US-9	TC-US9-04
FR-9	US-9	TC-US9-05

These tests validated the organization-side review process, including:

- Application retrieval for organization-owned projects
- Ownership authorization validation
- Application decision workflow (accept/reject)
- Prevention of invalid status transitions

Sprint 2 – Week 3 Tests (Workflow Completion & Business Rules)

Functional Requirement	User Story	Test Case ID
FR-8	US-8	TC-US8-03
FR-9	US-9	TC-US9-06
FR-9	US-9	TC-US9-07
FR-8	US-8	TC-E2E-01
FR-9	US-9	TC-E2E-02

Week 3 introduced tests validating system-level workflow integrity and business rule enforcement, including:

- Automatic rejection of competing applications when one is accepted
- Synchronization of application status across dashboards
- End-to-end workflow testing across system roles

6.2 User Acceptance Test Specification

The following acceptance tests validate the functionality implemented during Sprint 2 – Week 3, ensuring correct system behavior when business rules and cross-role workflows are applied.

TC-US8-03 – Student Dashboard Status Synchronization

Related Requirement: FR-8
User Story: US-8 – Track Application Status
Actor: Student

Preconditions:

- Student account exists and is authenticated.
- Student has submitted an application to a project.
- Application currently has status = pending.
- An organization user owns the associated project.

Steps:

1. Student logs into the system.
2. Student navigates to the My Applications Dashboard.
3. Organization user logs into the organization dashboard.
4. Organization accepts the student's application.
5. Student dashboard retrieves updated application data from backend.

Expected Result:

- HTTP 200 OK returned when retrieving student applications.
- Application status updates from Pending → Accepted.
- Status badge updates dynamically in the student dashboard.
- Badge color changes from yellow to green.
- No page refresh is required to display the updated status.

Postconditions:

- Application status remains **accepted** in the database.
- Student dashboard reflects the most recent application state.

TC-US9-06 – Prevent Multiple Accepted Applications

Related Requirement: FR-9
User Story: US-9 – Accept / Reject Applications
Actor: Organization

Preconditions:

- Organization user is authenticated.
- Organization owns the project.
- Multiple students have submitted applications.
- All applications initially have status = pending.

Steps:

1. Organization logs into the system.
2. Organization navigates to the project applications dashboard.
3. Organization selects one application and clicks **Accept**.
4. System processes the status update.

Expected Result:

- HTTP 200 OK returned.
- Selected application status updated to accepted.
- All other pending applications for the same project are automatically updated to rejected.
- Student dashboards reflect updated statuses.
- Organization dashboard displays only one accepted application.

Postconditions:

- Exactly one application remains accepted.
- All other applications associated with the project are rejected.
- System maintains data integrity across all related records.

TC-US9-07 – Reject Remaining Pending Applications Automatically

Related Requirement: FR-9
User Story: US-9 – Accept / Reject Applications
Actor: System Validation

Preconditions:

- Project has multiple pending applications.
- One application has already been accepted.

Steps:

1. System queries database for remaining applications associated with the project.
2. System identifies applications with status = pending.
3. System updates remaining applications.

Expected Result:

- All remaining pending applications transition to rejected.
- No additional applications may transition to accepted.
- Application state consistency maintained.

Postconditions:

- Project contains:
  - One accepted application.
  - Remaining rejected applications.
  - No pending applications remain.

TC-E2E-01 – End-to-End Student Application Workflow

Related Requirement: FR-7, FR-8, FR-9
User Story: US-6, US-7, US-8, US-9
Actor: Student / Organization

Preconditions:

- Student and organization accounts exist.
- Project listing exists.
- Student has not previously applied.

Steps:

1. Student browses project listings.
2. Student submits an application.
3. System stores application with status = pending.
4. Organization retrieves project applications.
5. Organization accepts the application.
6. Student dashboard retrieves updated application data.

Expected Result:

- Application successfully created.
- Application appears in organization dashboard.
- Organization decision successfully applied.
- Student dashboard displays updated status.
- Workflow completes without errors.

Postconditions:

- Application status = accepted.
- System state remains consistent across all dashboards.

TC-E2E-02 – Regression Testing for Sprint 1 Features

Related Requirement: FR-1 through FR-6
User Story: US-1 through US-5
Actor: System Validation

Preconditions:

- Sprint 2 features deployed.
- Existing Sprint 1 features remain accessible.

Steps:

1. User registers an account.
2. User logs into the system.
3. Student creates profile.
4. Organization posts project listing.
5. Student browses available listings.

Expected Result:

- All Sprint 1 functionality continues to operate correctly.
- No regression errors occur.
- Authentication, project creation, and browsing features remain functional.

Postconditions:

- System stability maintained.
- Previous sprint functionality unaffected.

6.3 Code Coverage Analysis

6.3.1 Overview

To ensure the reliability and correctness of the backend system implemented in Sprint 2, automated unit tests and integration tests were executed using pytest and coverage analysis tools. The objective of this testing effort was to verify that all implemented features, including application submission, organization decision workflows, dashboard synchronization, and role-based access control, operate correctly under expected conditions.

The coverage analysis evaluates how much of the backend source code was executed during the test suite. Higher coverage indicates that a larger portion of the system logic has been validated by automated tests.

The overall backend achieved 99% total code coverage, which demonstrates that nearly the entire codebase was exercised during testing.

Total Metrics

- **Total Statements:** 1350
- **Statements Missed:** 13
- **Overall Coverage:** 99%

This high level of coverage provides strong confidence that the system functions correctly and that key workflows are protected against regressions.

6.3.2 Models and Schema Coverage

Coverage: 100%

All database models and API schemas achieved full coverage.

Component	Coverage
Database models	100%
Application schema	100%
Project schema	100%
Student profile schema	100%
Organization profile schema	100%
User schema	100%

What This Confirms

The tests successfully verified

- Application model creation
- Foreign key relationships between students, organizations, and projects
- Database persistence logic
- Schema validation for request and response objects
- Data integrity across entities

Examples of validated behavior include

- Students submitting applications create valid records
- Projects correctly associate with organizations
- Application status values such as pending, accepted, and rejected persist correctly
- API schemas enforce required fields and data types

This ensures that data consistency and validation rules are fully protected by automated tests, which prevents malformed data from entering the system.

6.3.3 API Endpoint Coverage

Coverage: 96%

Router	Coverage
Applications	98%
Authentication	100%
Projects	95%
Student Profile	95%
Organization Profile	93%

API tests validated the main backend functionality including:

- Student application submission
- Organization viewing project applications
- Accepting and rejecting applications
- Retrieving projects and user profiles

Authentication endpoints reached 100% coverage, confirming that login, token generation, and protected route access were fully tested.

6.3.4 Role-Based Access Control

Coverage: 97%

Endpoint File	Coverage
dependencies.py	92%
auth router	100%

Role based access control ensures that

- Students can only access student endpoints
- Organizations can only manage their own projects
- Unauthorized users cannot modify protected resources

Tests verified

- Student authentication flows
- Organization authentication flows
- Access restrictions for protected routes
- Dependency injection for authenticated users

This confirms that the system enforces correct authorization policies across protected endpoints.

6.3.5 End to End Workflow Validation

In addition to unit tests, integration and end-to-end tests were executed to verify the complete application lifecycle.

Primary workflow test

- ➢ test_e2e_workflow.py

This test validated the full user interaction flow

1. Student browses project listings
2. Student submits an application

3. System stores application with pending status
4. Organization retrieves project applications
5. Organization accepts the application
6. Student dashboard retrieves updated application status

This directly verifies the requirement

- **TC E2E 01 End-to-End Student Application Workflow**

The test confirms that

- Applications are created successfully
- Organization decisions propagate correctly
- Student dashboards reflect updated statuses

6.3.6 Uncovered Code Analysis

Although coverage reached 99%, a small number of lines were not executed during testing.

File	Missing Lines
applications.py	1 line
organization_profile.py	3 lines
projects.py	2 lines
student_profile.py	2 lines
dependencies.py	2 lines

These uncovered lines correspond primarily to:

- Rare error-handling paths
- Defensive validation logic
- Edge cases that require abnormal inputs

Because these scenarios are difficult to trigger in standard workflows, they were intentionally deprioritized in favor of testing core system functionality.

6.3.7 Conclusion

The backend testing suite achieved 99% total code coverage, with full coverage of models, schemas, authentication logic, and most API endpoints. Core workflows such as application submission, organization decisions, and dashboard synchronization were thoroughly validated.

These results demonstrate a high level of confidence in the stability, correctness, and security of the backend system delivered in Sprint 2.

7. Source Code Development Progress

7.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

7.2 Summary of Contributions

Backend Contributions

- Implemented application workflow completion logic.
- Added **automatic rejection of competing applications when one application is accepted** to ensure that a project can only have one accepted applicant.
- Implemented validation preventing applications from being **updated after reaching a final state (accepted or rejected)**.
- Added additional validation handling for **non-existent application IDs**, ensuring that invalid requests return appropriate HTTP error responses.
- Expanded automated test coverage for application status transitions and workflow rules.
- Improved error handling and edge-case validation to increase backend reliability.

Frontend Contributions

- Enhanced the **Organization Applications Dashboard** to fully support the application decision workflow.
- Integrated the **Accept and Reject actions with backend API endpoints**, enabling organizations to update application statuses directly from the interface.
- Added **status badge updates and dynamic UI refresh behavior**, ensuring that status changes are immediately reflected in the dashboard.
- Implemented **navigation improvements within the Home dashboard**, allowing organizations to easily access the applications management interface.
- Improved overall dashboard usability and interface stability during testing.

7.3 Important Commits

Commit Message	Related Issue	User Story
feat: enforce single accepted application rule	#55	US-8
test: prevent multiple accepted applications	#55	US-8
ui: accept reject button workflow integration	#56	US-9
test: full integration with edge cases	#57	US-6 - US-9
test: e2e workflow	#57	US-6 - US-9

7.4 Burndown Chart

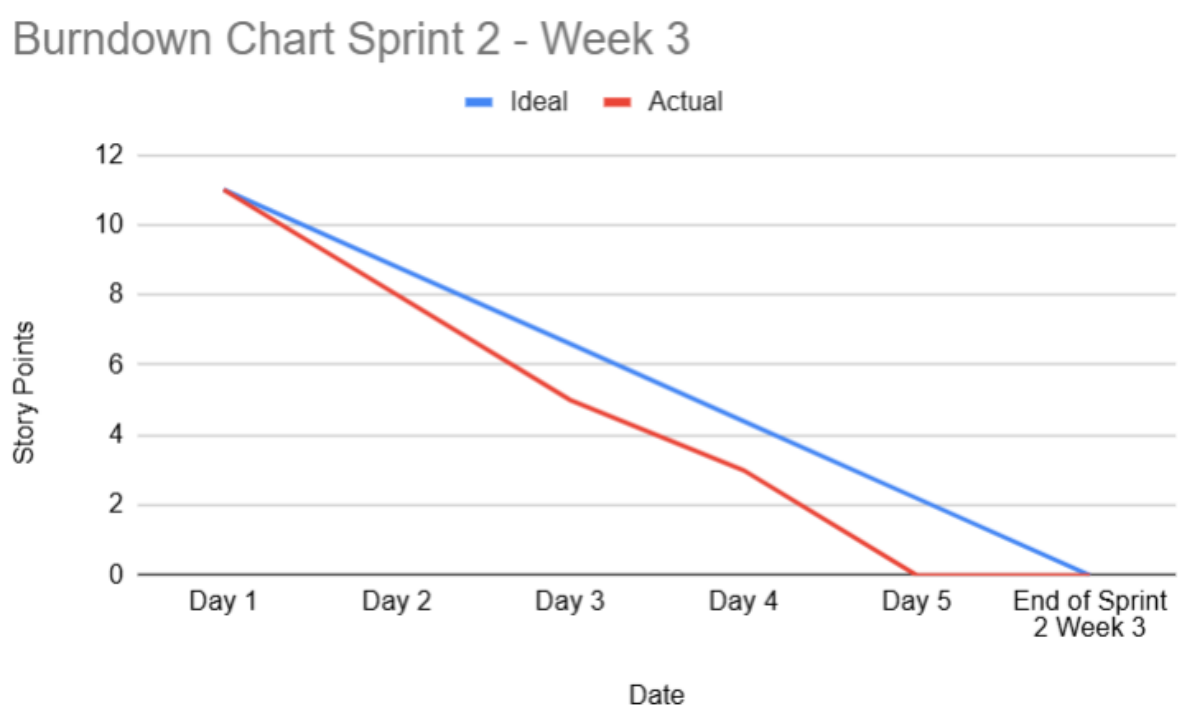

Figure 1 – Sprint 2 Week 3 Burndown Chart

Sprint Capacity: 11 Story Points
Sprint Duration: 5 Development Days

Burndown Chart Analysis

The burndown chart for Sprint 2 Week 3 reflects a validation-focused development cycle, where the primary effort involved completing workflow rules, enforcing business constraints, and stabilizing frontend interactions.

Unlike earlier sprint weeks that introduced entirely new system features, Week 3 concentrated on ensuring the correctness, stability, and integrity of the application decision workflow.

Day 1 – Workflow Rule Implementation

Progress during Day 1 focused on implementing core backend logic for application workflow completion.

The main development work involved enforcing the rule that only one student application can be accepted per project. Once an application transitions to the accepted state, all remaining pending applications must automatically transition to rejected.

Although story points were not fully burned during this phase, this work established the core logic necessary to maintain consistent system behavior and prevent conflicting application outcomes.

Day 2 – Status Transition Validation

Day 2 introduced additional validation rules governing application status updates.

The system was updated to prevent applications from being modified once they had reached a final state (accepted or rejected). This prevented invalid state transitions and protected data integrity.

The implementation of these safeguards resulted in the first major reduction in remaining work for the sprint.

Day 3 – Error Handling and Edge Case Coverage

During Day 3, development focused on strengthening error handling and ensuring robust system responses for invalid requests.

Additional validation was implemented to correctly handle scenarios where organizations attempt to update applications that do not exist in the system.

This included returning proper HTTP 404 responses for missing applications and expanding test coverage to verify these behaviors.

This stage helped ensure the backend API adhered to RESTful error handling standards and maintained predictable behavior for frontend integration.

Day 4 – Frontend Workflow Integration

Day 4 focused on integrating backend improvements with the frontend interface.

The Organization Applications Dashboard was enhanced to fully support the updated application workflow logic. Accept and Reject buttons were connected directly to the backend API endpoints, enabling organizations to perform decision actions directly within the interface.

Navigation improvements were also introduced on the Home page to allow organization users to easily access the applications dashboard.

This stage resulted in a noticeable drop in remaining work as frontend and backend components became fully synchronized.

Day 5 – Final Testing and System Validation

The final development day concentrated on end-to-end testing and UI stability validation.

Manual testing confirmed that the full application lifecycle behaved correctly:

- Students could apply to projects.
- Organizations could review submitted applications.
- Accepting an application automatically rejected competing applications.
- Application statuses were immediately reflected across dashboards.

Additional automated tests were executed to verify workflow rules and confirm that previous sprint functionality remained unaffected.

The sprint concluded with the remaining work reaching zero, indicating that all committed features and validation tasks were successfully completed.

Trend Interpretation

The burndown trend for Week 3 shows a steady reduction in remaining work with no scope expansion or unexpected task additions.

Key observations from the sprint progression include:

- No scope creep occurred during development.
- Backend workflow enforcement was completed before frontend integration.
- Validation and testing tasks were completed prior to final UI stabilization.

The development sequence followed a logical progression:

1. Workflow rule implementation
2. Status validation and error handling
3. Frontend integration
4. End-to-end testing and stabilization

This structured approach minimized integration risk and ensured that the system maintained consistent business logic across both backend and frontend components.

Velocity Alignment

Sprint 1 average velocity: ~16 points
Sprint 2 Week 1 completed: 17 points
Sprint 2 Week 2 completed: 13 points
Sprint 2 Week 3 completed: 11 points

The slightly lower point total during Week 3 reflects the nature of the work performed during this phase of the sprint. Rather than introducing large new features, the focus shifted toward workflow enforcement, validation logic, and system stabilization, which typically involve fewer story points but require careful implementation.

Despite the reduced point total, the sprint demonstrates continued stable and predictable velocity, indicating accurate estimation practices and effective sprint planning.

All committed work was completed within the planned five-day development window, reinforcing disciplined backlog management and sustainable development pacing.

8. Backlog Grooming

Sprint Backlog

All Sprint 2 issues completed.

Product Backlog Changes

- No new stories added.
- No reprioritization required.

Sprint 3 will begin with:

- Deliverables submission (US-10)
- Deliverables review (US-11)
- Completion workflow (US-12)

Backlog remains stable and aligned.

9. Final Sprint 2 Outcome

Sprint 2 delivered:

- Application submission
- Student visibility
- Organization review dashboard
- Accept/Reject workflow
- Strict transition enforcement
- Single-accept project rule
- 99% test coverage
- 100% passing test suite
- Zero unresolved defects

Sprint Goal: **Successfully Achieved**

10. Sprint Retrospective

The Sprint 2 Week 3 retrospective evaluates the development process, technical implementation, and collaboration outcomes during the sprint. I reflected on successes, challenges encountered, and actionable improvements that will strengthen future iterations of the project.

The retrospective discussion focused on three key questions:

1. What went well?
2. What did not go so well?
3. What can be improved in upcoming sprints?

10.1 What Went Well

Several aspects of the sprint were successful and contributed to steady development progress and overall sprint completion.

Strong Backend Architecture Implementation

One of the most positive outcomes of the sprint was the successful implementation of backend validation and business logic related to application decision handling. The Accept/Reject endpoint required careful design to ensure that applications transitioned only between valid states, and that organizations could only modify applications belonging to their own projects.

The backend implementation successfully enforced:

- Organization ownership validation
- Role-based access control (RBAC)
- Valid application status transitions
- Prevention of invalid or duplicate updates

These safeguards improved the overall reliability of the application system and ensured that application states remained consistent throughout the platform.

Clear API Contracts Between Frontend and Backend

Another major success was the establishment of clear API endpoints and response formats between the backend and frontend components. By finalizing the backend

endpoints first, the frontend implementation was able to integrate smoothly with minimal rework.

For example, once the endpoint for retrieving project applications was finalized, the frontend dashboard could immediately consume the data and render applicant information correctly. This sequencing significantly reduced integration issues.

Improved Test Coverage

I also made significant improvements in testing during this sprint. Instead of only testing basic functionality, additional test cases were written to validate edge cases and error handling scenarios.

Examples include:

- Verifying that unauthorized users cannot view or modify applications
- Ensuring that application status updates fail if the application does not exist
- Confirming that only pending applications can transition to accepted or rejected

This expanded test coverage increased confidence that the system behaves correctly under both normal and unexpected conditions.

Successful Frontend Integration

The frontend implementation of the Organization Applications Dashboard worked well and successfully integrated with the backend APIs. The dashboard now allows organizations to:

- View all applicants for their projects
- Navigate through applications using pagination
- Accept or reject applications directly from the interface
- Confirm decisions through a modal dialog to prevent accidental actions

These UI improvements significantly enhanced usability and demonstrated that the backend functionality could be effectively exposed through the user interface.

Stable Sprint Scope

Another positive aspect of the sprint was the stability of the sprint backlog. No additional stories were introduced mid-sprint, and I remained focused on completing the originally

committed tasks. This helped maintain development momentum and prevented disruptions to the planned workflow.

10.2 What Did Not Go So Well

Although the sprint was completed successfully, several challenges and inefficiencies were encountered during development.

Initial Uncertainty Around State Transition Logic

One of the more difficult aspects of the sprint was implementing the correct logic for application status transitions. At the beginning of development, there was some ambiguity about how strictly the system should enforce status changes.

For example, questions arose such as:

- Should an already accepted application be allowed to change back to pending?
- Should rejected applications be allowed to become accepted later?
- What happens if an organization accidentally attempts to update an application twice?

These uncertainties caused minor delays while the correct business rules were clarified and implemented.

Frontend–Backend Integration Timing

Although the integration between frontend and backend ultimately worked well, there were moments where the frontend could not be fully tested because backend endpoints were still being finalized.

This created short periods where frontend progress slowed while waiting for the backend logic to stabilize.

While this did not significantly impact the sprint timeline, it highlighted the importance of clearly sequencing development tasks so that dependencies are addressed earlier.

Debugging and Validation Complexity

Another challenge during this sprint was debugging validation errors related to role-based permissions and project ownership checks.

Because several conditions needed to be validated simultaneously, it was sometimes difficult to determine which validation rule caused a request to fail. This required additional debugging and logging to isolate the issue.

Although these problems were eventually resolved, they increased development time and highlighted areas where clearer error handling and logging could improve efficiency.

Limited Early Testing During Development

Testing was conducted primarily after major features were implemented, rather than continuously throughout development. As a result, a few small issues were discovered later in the sprint that could have been detected earlier with more frequent incremental testing.

While these issues were relatively minor, they demonstrated the benefit of integrating testing earlier in the development process.

10.3 What Can Be Done Better

Based on the challenges encountered during the sprint, I identified several concrete improvements that will help strengthen future development cycles.

Define Business Rules Earlier

To avoid confusion around application state transitions, I plan to define all business rules and state diagrams earlier in the sprint planning phase. This will ensure that developers clearly understand the expected system behavior before implementation begins.

For example, future sprint planning will include:

- Explicit documentation of allowed status transitions
- Clear definitions of invalid state changes
- Examples of expected system responses

This preparation will reduce ambiguity and prevent implementation delays.

Adopt Incremental Testing Practices

To improve development efficiency, testing will be performed earlier and more frequently during feature implementation rather than waiting until the end of the sprint.

Specifically, I will:

- Write test cases alongside feature development
- Validate endpoints immediately after implementation
- Perform small regression tests after each major change

This approach will help identify bugs sooner and reduce the amount of debugging required later.

Improve Error Logging and Debugging Support

During this sprint, some validation failures were difficult to diagnose because the error messages were not always specific enough.

To address this issue, future development will include:

- More detailed backend logging
- Clearer API error messages
- Additional debug information during development builds

These improvements will make troubleshooting faster and reduce development friction.

Better Task Sequencing for Frontend and Backend Work

Although the integration ultimately succeeded, I recognized that frontend development occasionally depended on backend features that were still being finalized.

To improve workflow efficiency, future sprints will prioritize completing backend API endpoints earlier in the sprint so that frontend developers can integrate with stable interfaces sooner.

This sequencing will reduce waiting periods and improve parallel development.

Continue Expanding Test Coverage

Finally, I plan to continue expanding automated testing coverage to include more edge cases and error conditions.

Additional tests will focus on:

- Invalid authentication scenarios
- Unauthorized access attempts

- Boundary cases for pagination
- Data validation errors

Increasing test coverage will strengthen system reliability and reduce the risk of regressions in future updates.

10.4 Retrospective Summary

Sprint 2 Week 3 was successful and resulted in the completion of all committed user stories and technical tasks. The sprint demonstrated strong progress in backend validation, frontend integration, and system testing.

While some challenges occurred related to validation logic and debugging complexity, I successfully resolved these issues and identified clear strategies to improve development efficiency in future iterations.

By implementing earlier business rule clarification, more incremental testing, improved error logging, and better task sequencing, I expect future sprints to become even more efficient and predictable.

The lessons learned during this retrospective will guide the planning and execution of upcoming development cycles and support the continued growth of the MicroMatch platform.